Harness-Controlled LLM Autonomous Research: An Experimental Governance Framework Based on the Session, Sandbox, and Harness Layers

Pei Xu

iFLYTEK; China; sheldum03@gmail.com

Abstract. As LLM applications mature in software development and scientific experimentation, autonomous research is gradually moving from human-led experiment planning toward model-driven automatic closed-loop optimization. However, under fixed-budget and long-horizon settings, LLM agents are prone to historical forgetting, duplicate proposals, direction drift, and insufficient reproducibility. This paper proposes a harness control framework for parameter-controlled autonomous experiments. The system is decomposed into three clearly scoped layers: the Session layer, the Sandbox layer, and the Harness layer. Three external mechanisms - Memory Retrieval, Experiment Deduplication, and Heartbeat Summarization - are designed to improve state governance.

Using a 4090D single-GPU parameter-controlled compatibility verification task as the benchmark, this paper conducts empirical evaluation under a parameter-restricted Formal AutoResearch setting, where the agent may submit only whitelisted hyperparameters. The experimental setup covers five method groups across baseline, single mechanisms, and full three-mechanism control, with 5 random seeds for each method and 20 completed rounds for each method-seed combination, all under a unified model and unified resource constraints. The results show that, in this parameter-controlled compatibility task, external state-governance mechanisms change agent exploration behavior: Memory obtains the lowest mean best_val_bpb, Dedup is the most stable in seed-paired comparison and intercepts duplicate parameter proposals, and Full harness does not show monotonic additive gain. These findings indicate that, in autonomous experiments with a formalizable parameter space and fixed training program, performance is affected not only by model reasoning ability but also by the external state-governance structure. Engineering the layering of experiment memory, execution, and decision-making is a feasible path to improving controllability in such long-horizon autonomous experiments.

Keywords: LLM agents; autonomous research; experiment deduplication; memory retrieval; heartbeat summarization; reproducibility; compute-budget constraints; closed-loop governance

1. Introduction

1.1 Research Background

In recent years, LLMs have demonstrated significant capability in software generation, problem diagnosis, and experiment recommendation. AutoResearch-style methods form a closed loop for advancing research automatically through the process of proposing modifications, executing training, evaluating feedback, and iterating backward. This paradigm reduces human intervention and accelerates experimental iteration, but engineering practice exposes a common tension: the model itself may be powerful, while the autonomous process remains unstable.

This instability usually appears in three forms:

- **State Degradation: as experiment rounds accumulate, key experience is gradually buried by new logs in the context.**
- **Redundant Trials: under discrete hyperparameters and prompt variation, the model repeatedly explores regions that have already been tested.**
- **Tangled Execution: experiment-management logic and proposal logic are mixed in the context, making behavior difficult to audit.**

Under a fixed budget, these three issues amplify one another. Once duplicate proposals appear, GPU time is consumed ineffectively; once direction drift appears, the best improvement point is delayed; once logs become disordered, reproduction and mechanism evaluation are both obstructed. In practice, many teams focus on stronger models or longer contexts while overlooking the governance infrastructure of the autonomous system.

1.2 Research Questions

This paper focuses on the following question: given a fixed task and model, how can external control mechanisms improve the efficiency and stability of parameter-controlled autonomous experiments without changing the expressive capacity of the agent model? More specifically, we need to answer:

1. How can the model accurately absorb prior experience before each decision round and avoid losing critical context?
2. How can duplicate proposals be identified quickly within the parameter space to avoid budget waste?

3. How can state information be compressed periodically so that long-horizon decisions remain directionally consistent?
4. Do these mechanisms produce interactions or costs when used alone and in combination?

1.3 Objectives and Structure

The objective of this paper is to build a reproducible, interpretable, and extensible control framework for a parameter-controlled compatibility task and to quantify its effect through controlled experiments. The paper is organized as follows:

- Section 2 reviews related work and gaps;
- Section 3 presents the problem formulation;
- Section 4 describes the three-layer architecture and three control mechanisms;
- Section 5 presents the experimental setup and metrics;
- Section 6 reports the results and analyzes the mechanisms;
- Section 7 discusses mechanism behavior and limitations;
- Section 8 concludes and outlines future work.

1.4 Terminology and Notation

To ensure consistency throughout the paper, we use the following terms:

- **Agent: a single LLM-based agent that generates training proposals.**
- **Session: structured storage that persists historical proposals and experiment results.**
- **Sandbox: the execution layer that runs training, collects logs, and isolates the environment.**
- **Harness: the scheduling layer responsible for proposal legalization, deduplication, decisions, summarization, and state updates.**
- **Completed Round: an execution round that finishes one valid training run and produces usable metrics.**
- **Scheduled Round: a scheduling attempt that includes proposal retries.**
- **Duplicate Proposal: a proposal whose effective parameter signature is exactly identical to historical records.**
- **keep/discard/crash: the three decision classes: keep, discard, and execution failure.**

1.5 Contributions

The contributions of this paper are as follows:

5. We propose and implement a clearly layered autonomous-experiment control framework: Session/Harness/Sandbox.
6. We design three composable external mechanisms - memory retrieval, deduplication, and heartbeat summarization - and expose them through a unified interface.
7. We establish a reproducible evaluation protocol on a 4090D parameter-controlled compatibility task and compare performance and process metrics under multiple mechanism configurations.
8. Positioned as an engineering-governance empirical study, this paper observes that the mechanisms do not combine linearly and emphasizes task-dependent control-strategy selection.

2. Related Work

2.1 AutoResearch and the Autonomous Experiment Loop

AutoResearch-style systems place an LLM agent in a closed loop of proposing modifications, executing training, evaluating results, and keeping or rolling back. Karpathy's AutoResearch emphasizes a single GPU, a fixed time budget, and unattended iteration: the agent modifies training code, runs short training jobs, and keeps or discards modifications according to validation metrics [13]. The value of such systems lies in transforming research trial-and-error into a continuously executable experiment loop, but their original form mainly relies on prompts, logs, and version control to maintain state, without systematically separating the three responsibilities of historical fact recording, execution isolation, and governance decisions.

AI Scientist further extends autonomous research into an end-to-end workflow covering idea generation, code implementation, experiment execution, chart generation, paper writing, and automatic review [12]. Later, AI Scientist-v2 improves open-ended research generation through agentic tree search and an experiment-management agent [18]. These works show that LLMs can participate in a more complete research pipeline, but their main focus is open-ended discovery and paper production. In contrast, this paper does not attempt to prove a general automatic scientist; instead, it narrows the object of study to a parameter-controlled AutoResearch compatibility task and examines the auditable effect of external governance mechanisms under a fixed budget, fixed training program, and fixed parameter space.

2.2 Agent Memory, Reflection, and Long-Horizon Context

Research on LLM agents has proposed multiple approaches to long-horizon context management. ReAct advances tasks through alternating reasoning and action in response to external environmental feedback [3]; Generative Agents use memory streams, reflection, and retrieval to support long-term behavioral consistency [6]; Reflexion transforms task feedback into linguistic reflections and stores them in episodic memory for later trial-and-error [14]; MemGPT models context management as a multi-level memory scheduling problem inspired by operating-system virtual memory [15]; Voyager accumulates reusable code snippets through a skill library in the Minecraft environment and continues exploration by combining those skills with environmental feedback [16].

The common point of the above research is to help agents remember more or reflect from feedback. In autonomous experimentation, however, text memory alone is insufficient: experiment history requires computable parameter signatures, failure types, keep/discard/crash decisions, seed-level calibration values, and budget state. The memory retrieval in this paper is not general-purpose RAG; instead, it models the experimental object as a structured session record so that later proposals can consume auditable experimental facts.

2.3 Experiment Management, Failure Tracking, and Reproducibility

Experiment-management systems such as MLflow can record parameters, metrics, model artifacts, and runtime metadata [10], providing visualization and tracking capabilities for machine-learning experiments. In LLM autonomous experiments, however, the logging system itself does not determine how the agent acts in the next round. If the decision layer cannot directly consume failure semantics, duplicate judgments, and the current best state, logs are only post hoc audit material rather than closed-loop control signals.

Therefore, this paper moves experiment management from result recording to pre-decision governance: the session layer provides facts, the sandbox layer guarantees execution isolation, and the harness layer transforms facts into proposal filtering, deduplication, summarization, and state updates. This positioning is closer to engineering governance than to a simple experiment dashboard.

2.4 AutoML, Hyperparameter Search, and Deduplication Governance

Random search, Bayesian optimization, Hyperband, and related methods have shown that sampling strategy, early stopping, and budget allocation can significantly affect search efficiency in structured hyperparameter spaces [7-9]. AutoML systems further automate

model selection, preprocessing, hyperparameter optimization, and ensemble learning [17]. These methods usually assume that the search space and optimizer are explicitly defined by an algorithm, so repeated points or low-value points can be handled directly by the scheduler.

LLM autonomous experiments differ because proposals are produced by a natural-language agent, and parameter choices are affected by prompts, historical summaries, and feedback wording. Even if the final parameter space is a discrete whitelist, the agent may still repeatedly propose combinations that have already been tested. This paper transfers the search-space governance idea from AutoML into the LLM-agent closed loop, generates signatures from complete effective parameter vectors, and moves exact duplicate interception before training.

2.5 Multi-Agent Collaboration and Control-Layer Infrastructure

Multi-agent systems improve complex task handling through role division, such as decomposing research tasks into planning, coding, review, and analysis roles. Multi-agent designs can alleviate the limitations of a single agent, but they do not automatically solve long-horizon state governance: without a unified session bus, failure classification, and budget constraints, multiple agents may still share inconsistent histories, repeat trials, or amplify contextual noise.

This paper uses a single main agent in order to isolate the effect of external control mechanisms. The Session/Harness/Sandbox layering does not exclude multi-agent systems. On the contrary, it provides a reusable fact layer and governance layer for multi-agent systems. If multiple agents are introduced later, different roles can share the same session record and deduplication constraints instead of each maintaining unauditable local memory.

2.6 The Gap Addressed by This Paper

In summary, prior work has demonstrated four kinds of capability: AutoResearch shows that LLMs can execute experiments in a closed loop; AI Scientist shows that LLMs can organize a more complete research pipeline; agent memory and reflection mechanisms show that long-horizon context can improve agent behavior; and AutoML and experiment-tracking systems show that structured search and recording are important for machine-learning experiments. The gap filled by this paper is not a stronger model or a more complex automatic scientist, but a reproducible empirical comparison of memory, deduplication, and heartbeat summarization as external governance mechanisms in a parameter-controlled AutoResearch task.

Thus, the best positioning of this paper is as an engineering-governance empirical study: it focuses on state structure, execution isolation, duplicate-proposal interception, and result auditability under a fixed budget, and uses the 4090D parameter-controlled compatibility task to verify how these mechanisms change agent exploration behavior.

3. Problem Formulation and Objectives

3.1 Closed-Loop Formalization

Define the system state at round t:

\[S_t = (C_t, H_t, b_t^*, v_t) \] where:

- *C_t denotes execution constraints, including the hyperparameter whitelist, timeout, device, and budget;*
- *$H_t = \{(p_i, r_i, d_i)\}_{i=1}^{t-1}$ denotes the historical record;*
- *b_t^* denotes the current best validation metric up to round t; lower is better;*
- *v_t denotes the session-state version.*

In the formal experiment, b_0^* is initialized from the baseline reference of the same random seed. This reference is the unmodified starting calibration value and is used to interpret later Δ values and keep/discard decisions; it is not the same as the Baseline method group.

The main agent generates proposal p_t from the retrieved context $O_t\subseteq H_t$. The control layer normalizes it into an execution mapping e_t, receives result r_t after sandbox execution, and makes decision d_t:

\[d_t = \mathcal{D}(S_t,p_t,r_t),\quad d_t\in\{\text{keep},\text{discard},\text{crash}\} \]

Define the best-state update:

\[b_{t+1}^* = \begin{cases} \min\left(b_t^*, r_t^{\text{val_bpb}}\right), & d_t=\text{keep} \\ b_t^*, & d_t\in\{\text{discard},\text{crash}\} \end{cases} \]

3.2 Optimization Objective

Under a fixed completed-round budget N, the optimization objective is:

\[\min\ b_N^* = \min\left(b_0^*, r_1^{\text{val_bpb}}, \ldots, r_N^{\text{val_bpb}}\right) \] while constraining the failure rate, duplicate rate, and reproducibility to remain controllable.

Unlike approaches that only examine the best value, this paper also minimizes ineffective search overhead.

3.3 Evaluation Metrics

This paper divides evaluation into four categories:

9. **Effectiveness metrics: final best_val_bpb and the best-so-far curve.**
10. **Efficiency metrics: GPU minutes per Keep.**
11. **Stability metrics: crash rate, duplicate-proposal rate, and scheduled-round inflation.**
12. **Governance metrics: duplicate retry exhaustion count, skipped rounds, and session-record completeness.**

3.4 Task Boundary and External Validity

This experiment uses a parameter-restricted task, allowing the proposal space to be formalized as discrete combinations. This setting helps analyze duplicate proposals and control mechanisms precisely, but it also means that the results cannot be directly generalized to arbitrary code-refactoring tasks. The discussion section further states this boundary.

4. Method

4.1 Three-Layer Architecture

Figure 1 Three-layer autonomous research architecture. The session layer maintains global experiment memory; the sandbox layer is responsible for stable execution; the control layer provides governance between them.

Figure 1. Three-Layer Autonomous Research Architecture under Harness Control

The architectural points are as follows:

- The session layer is the fact layer: it records experiment trajectories and failure semantics;
- The sandbox layer is the execution layer: it guarantees experiment reproducibility;
- The control layer is the governance layer: it performs decision filtering, deduplication, summarization, and updates.

This separation lets the model focus only on generating proposals, while complex and inefficient engineering tasks are handled uniformly in the control layer. This paper adopts a decoupled design consisting of a single Proposal Agent and harness-side deterministic controllers: the LLM is responsible only for generating the next-round parameter proposal;

memory retrieval, parameter deduplication, heartbeat summarization, failure recording, and budget control are all executed by local harness code. This design avoids the additional randomness and cost introduced by multi-agent collaboration and also makes the experiment process easier to reproduce and audit.

4.2 Session-Layer Design

4.2.1 Structured Entries

Each completed round writes the following fixed fields:

- experiment_id
- proposal_signature
- param_map
- status
- metrics, including val_bpb, time, VRAM, and step count
- failure_reason
- tags

The session layer uses an append-only strategy and retains historical versions, avoiding distortion caused by posterior overwriting. Later retrieval and deduplication can rely on this structure to run quickly.

4.2.2 Retrieval Index

To improve retrieval efficiency, lightweight indexes are built by direction tag, time, and signature:

- recent-success index, ordered by round in reverse;
- recent-failure index;
- failure-type index, such as OOM, NaN, and timeout;
- parameter-signature index for fast deduplication.

4.3 Sandbox-Layer Design

The goal of the sandbox layer is execution determinism. Its responsibilities are as follows:

- Use an independent directory for each run to avoid residual contamination;
- Execute a unified script after enforced parameter mapping;
- Capture logs uniformly and identify error types;

- Return metrics and exceptions in a fixed format.

The sandbox does not interpret experimental semantics; instead, it passes results to the control layer for judgment.

4.4 Control Layer: Memory, Deduplication, and Summarization

4.4.1 Memory Retrieval

At the beginning of each round, the control layer extracts from the session layer:

- historical successful paths, including key parameters;
- recent failure directions and forbidden regions;
- trend summaries from the recent k rounds.

The retrieval results are compressed into a fixed template and injected into the main-agent context, reducing contextual noise.

> *Example: successful direction: `EMBD` up + medium-speed `LR` combinations once improved continuously; forbidden direction: `SEQ_LEN=256` produced NaN repeatedly; priority recommendation: fine-tune the combination of batch and LR, and do not repeat past failed parameters.*

This process turns unstructured logs into decision-ready context.

4.4.2 Experiment Deduplication

Each proposal is first standardized into a complete effective parameter vector θ, then a signature is generated:

\[\mathrm{sig}(p)=(\text{target},\text{mechanism},\text{operation},\theta) \] where θ is expanded in a fixed field order to avoid false judgments caused by missing fields.

The deduplication judgment is as follows:

- **exact_duplicate: the parameter signature is exactly identical.**
- **near_duplicate: changes in core parameters are below a threshold.**
- **related: parameters under the same target have structural differences.**
- **novel: low relevance to history, executable.**

This experiment enables exact_duplicate as the formal statistical criterion: when parameter signatures are exactly identical, the proposal is rejected and regenerated; near_duplicate and semantic-level duplicates are treated as boundaries for later extension and are not included in the main results of this round.

A retry cap R is set; once it is exceeded, the scheduled round is marked as skipped to keep the completed-round budget consistent.

4.4.3 Heartbeat Summarization

Heartbeat trigger conditions:

- Every 5 completed rounds;
- 3 consecutive discards;
- A new best appears;
- A key failure such as crash, OOM, or NaN occurs.

Each summary entry contains four parts: 1. current feasible directions; 2. rejected directions; 3. unverified hypotheses; 4. next-step priorities.

Summarization does not replace decisions; it periodically archives and redirects the state.

4.5 Decision Strategy and State Update

Each seed first runs a baseline reference and obtains the unmodified starting point b_0^* for that seed. Given validation result r_t and current best b_t^*:

$$ d_t=\begin{cases} \text{keep}, & r_t < b_t^* \\ \text{discard}, & r_t \ge b_t^* \\ \text{crash}, & \text{execution failure} \end{cases} $$

The update rule is:

$$ b_{t+1}^*=\begin{cases} \min(b_t^*,r_t^{\text{val\_bpb}}), & d_t=\text{keep} \\ b_t^*, & d_t=\text{discard or crash} \end{cases} $$

4.6 Complexity Discussion

The additional overhead of the control layer can be divided into three parts:

- Retrieval: $O(\log n)$ or constant-time index query;
- Deduplication: $O(1)$ hash lookup;
- Summarization: a small amount of text-generation overhead.

Compared with minute-level training time per round, control-layer overhead is negligible and improves overall experiment utilization.

4.7 Overall Algorithm

Algorithm 1. Harness-Controlled AutoResearch Loop

```
Input: dataset, seed, N, dedup_cap, heartbeat_k
Load baseline code, fixed randomness seeds
baseline_ref = Sandbox.run_baseline(seed)
Initialize Session, metrics, best = baseline_ref.val_bpb, completed = 0

while completed < N:
  # 1) Read context
  ctx = Session.retrieve(
    top_success, top_fail, recent_trend, blocked_directions
  )
  proposal = Agent.generate(ctx)

  # 2) Proposal validation and deduplication
  valid = false
  attempts = 0
  while attempts < dedup_cap and not valid:
    attempts += 1
    if not Validator.whitelist(proposal):
      Session.log_invalid(attempts, proposal)
      proposal = Agent.regenerate(ctx)
      continue

    effective = Validator.apply_defaults(proposal)
    label = Dedup.classify(effective, Session)
    if label == exact_duplicate:
      Session.log_dup(label, attempts, effective)
      proposal = Agent.regenerate(ctx)
      continue

    valid = true

  if not valid:
    Session.log_skip_round("duplicate_param_retry_exhausted")
    continue

  # 3) Training execution
  result = Sandbox.run(effective)
  status = Evaluate(result, best)

  # 4) Decision update
```

```
    if status == keep:
        completed += 1
        best = min(best, result.val_bpb)
        Session.update_best(result)
    elif status == discard:
        completed += 1
    else:
        completed += 1

    Session.append(proposal, effective, result, status)

    # 5) Heartbeat summarization
    if Heartbeat.triggered():
        Session.append(Heartbeat.summary(Session))
```

Algorithm 1. The controller first performs memory injection, then validation and deduplication, followed by training execution and decision update, with heartbeat summarization performed when trigger conditions are met.

5. Experimental Design

5.1 Experimental Task and Environment

This study uses a single-GPU compatibility verification task as the main task. The main parameters include embedding scale, batch size, sequence length, learning rate, and weight decay. To ensure fair comparison:

- the training program is fixed;
- data preprocessing is fixed;
- the agent model is fixed;
- each completed round is constrained to 120 s;
- 5 random seeds are used.

This frozen experiment ran under the RTX 4090D parameter-controlled protocol, with DeepSeek-V4-Flash as the agent model, patch_mode=param_only, and param_space=compat. The main results come from the frozen summary files of runs_4090d_formal_12g_param, with summary.json, summary.csv, summary.md, and the SHA256 checksum of the archive package retained.

The reproducibility configuration checklist is as follows:

Item	Setting
Hardware	Single RTX 4090D
Agent model	DeepSeek-V4-Flash
Code snapshot	`test-4090` branch, commit `b6df87d`
Protocol	`patch_mode=param_only`, `param_space=compat`
Training program	Fixed compatibility training program; the agent submits only whitelisted hyperparameters through environment variables
Method groups	Baseline, Memory, Dedup, Heartbeat, Full
Random seeds	1, 2, 3, 4, 5
Budget	20 completed rounds per method-seed episode, 500 completed rounds in total
Time limit	Training limit of 120 s per completed round
Starting calibration	Each seed uses one baseline reference to initialize b_0^* and calculate `delta_vs_baseline`
Frozen directory	`runs_4090d_formal_12g_param`
Archive checksum	`runs_4090d_formal_12g_param_full500_20260617-102948.tar.gz`, SHA256=`644bab59f961dd43543453677b78b9dad8dc0412ec90aa3b6b749aace08fc16b`

5.2 Control Groups and Methods

Table 1 Method configuration. The baseline keeps only concise summaries of the most recent five rounds and introduces none of the three external mechanisms.

Method	Memory	Dedup	Heartbeat
Baseline	×	×	×
+Memory retrieval	✓	×	×
+Experiment deduplication	×	✓	×
+Heartbeat summarization	×	×	✓
Full control (Full)	✓	✓	✓

5.3 Parameter-Controlled Strategy

The whitelist parameters are defined as:

- AUTORESEARCH_COMPAT_EMBD: 32, 64, 128, 256, 512
- AUTORESEARCH_COMPAT_BATCH_SIZE: 16, 32, 64, 128
- AUTORESEARCH_COMPAT_SEQ_LEN: 64, 128, 256
- AUTORESEARCH_COMPAT_ADAMW_LR: 0.003, 0.001, 0.0005, 0.0001
- AUTORESEARCH_COMPAT_ADAMW_WEIGHT_DECAY: 0.1, 0.01, 0.0

Out-of-range proposals are directly judged invalid and enter the regeneration process.

5.4 Budget and Reproducible Experiment Setup

Each method covers 5 random seeds, and each method-seed episode is fixed to 20 completed rounds, for a total of 100 completed rounds per method. The overall target is 500 completed rounds. Considering failed retries, scheduled rounds differ from completed rounds; duplicate parameter proposals are proposal-layer retries, and only retry exhaustion is recorded as a skipped round. All settings are fixed to the same commit and frozen archive to avoid incidental differences.

5.5 Evaluation Metrics and Recording Specification

5.5.1 Result Metrics

- **best_val_bpb_after_N: the best value for each method-seed after N=20 completed rounds.**
- **best-so-far curve: shows the trajectory of the best value over 20 completed rounds.**

5.5.2 Process Metrics

- duplicate-proposal rate
- retry-exhaustion count
- skipped rounds
- GPUmin/keep

5.5.3 Robustness

Report mean +/- standard deviation and provide seed-level behavior. This paper does not exaggerate conclusions based on a single lowest value, in order to avoid chance effects.

5.6 Documents and Figures

The experimental results correspond to the following figures and tables:

- **Figure 1: three-layer architecture diagram;**
- **Figure 2: best-so-far curve comparison;**
- **Figure 3: duplicate parameter proposal interception comparison;**
- **Figure 4: Keep-ratio comparison;**
- **Table 2: main-metric comparison;**
- **Table 3: seed-paired comparison relative to the Baseline method;**
- **Table 4: process-metric comparison;**

- **Algorithm 1: control-flow pseudocode.**

6. Experimental Results

6.1 Overall Method Performance

Method	Mean final val_bpb (down)	Standard deviation	Improvement relative to baseline reference (up)	Keep ratio (up)	Crash rate (down)
Baseline	2.482567	0.004518	0.021201	0.150000	0.000000
+Memory retrieval	**2.472628**	0.017415	**0.031140**	**0.180000**	0.000000
+Experiment deduplication	2.474095	0.007637	0.029673	0.130000	0.000000
+Heartbeat summarization	2.483234	0.005049	0.020534	0.110000	0.000000
Full control (Full)	2.482035	0.009853	0.021733	0.110000	0.000000

Table 2 Performance metrics (5-seed average).

The results show that memory retrieval performs best in final mean val_bpb, followed closely by the deduplication mechanism. Here, improvement relative to the baseline reference refers to the difference between each seed's baseline reference and that method's best_val_bpb; it is not the same as the paired difference relative to the Baseline method. The numbers indicate that:

- The memory mechanism has the best mean, but seed-level differences are large;
- Although the deduplication mechanism does not always improve the keep ratio, it directly records and intercepts duplicate parameter proposals at the proposal layer.

Table 3 Seed-paired comparison relative to the Baseline method. The difference is defined as best_val_bpb(method)-best_val_bpb(baseline); a negative value indicates that the method outperforms Baseline. The interval is a deterministic bootstrap descriptive interval for the five seed-paired differences and is not treated as a formal significance test.

Method	Paired mean difference (down)	Bootstrap 95% CI	Seeds beating Baseline	Seeds worse than Baseline
Baseline	0.000000	[0.000000, 0.000000]	0	0
+Memory retrieval	-0.009940	[-0.026709, 0.004907]	3	2
+Experiment deduplication	**-0.008472**	**[-0.013896, -0.003522]**	**5**	**0**
+Heartbeat summarization	0.000666	[-0.003391, 0.004286]	2	3
Full control (Full)	-0.000533	[-0.006520, 0.005504]	3	2

Figure 2 best-so-far val_bpb curves for different methods over 20 completed rounds.

Figure 2. Best-so-far val_bpb over 20 rounds

6.2 Process-Metric Analysis

Method	Proposal count	Duplicate proposals	Retry exhaustion	Skipped rounds	GPUmin/keep (down)
Baseline	100	0	0	0	22.321604
+Memory retrieval	100	0	0	0	16.234592
+Experiment deduplication	168	68	0	0	19.619830
+Heartbeat summarization	100	0	0	0	25.025218
Full control (Full)	219	119	1	1	18.944104

Table 4 Process metrics.

A readability-oriented interpretation is as follows: 1. the deduplication group and full-control group produce more retry attempts at the proposal layer, which is the cost of exploration completeness; 2. memory retrieval has no duplicate blocking within completed rounds and has the lowest GPUmin/keep; 3. the high proposal count of full control reflects its denser control chain, but keep efficiency does not improve linearly, indicating that its strategy is conservative.

Figure 3 Duplicate parameter proposal interception in the deduplication group and full-control group.

Figure 3. Duplicate parameter proposals

Figure 4 Keep-ratio comparison across methods.

Figure 4. Keep ratio by method

6.3 Mechanism Analysis

6.3.1 Benefits of Memory Retrieval

Memory retrieval compresses historical successful paths, failure directions, and recent trends and injects them into the context. In this experiment, its direct manifestation is the lowest mean best_val_bpb and the lowest GPUmin/keep; however, because its seed-level variance is large, it is more appropriate to state that it improved some exploration trajectories rather than proving stable effectiveness across all seeds.

6.3.2 Value and Boundaries of Deduplication

The deduplication mechanism has a clear statistical criterion in parameter-restricted tasks because proposals can be mapped to complete effective parameter signatures. Rejecting exact duplicates moves duplicate risk to the pre-proposal stage, thereby preserving the training budget for exploration of new signatures. This conclusion currently covers only parameter-level duplication, not more complex semantic duplication.

6.3.3 Governance Value of Heartbeat Summarization

Heartbeat summarization emphasizes process stability: when consecutive failures or a new best appears, it explicitly records direction-switching logic and reduces agent drift by intuition in long contexts. Its benefit is not necessarily reflected directly in the final best value, but in interpretability and long-task operability.

6.4 Representative Cases

6.4.1 Baseline's Repetition Defect

When the baseline does not perform signature deduplication, the control layer does not record or intercept duplicate parameter signatures before proposal execution. Per-round records show that Baseline returns multiple times to similar medium-scale parameter combinations; these attempts still enter actual training and are judged by keep/discard only after training. This phenomenon shows that the baseline lacks a mechanism for moving repeated or near-repeated attempts to pre-execution filtering, but it is not treated as identical to the exact duplicate statistic already reported.

6.4.2 Dedup Interception Path

The Dedup group recorded 68 duplicate parameter proposals, and the Full harness group recorded 119 duplicate parameter proposals. The deduplication layer marks exact_duplicate at the proposal stage and regenerates proposals, so these duplicate signatures do not consume completed training rounds. The full-control group had 1 duplicate retry exhaustion event

recorded as a skipped round, showing that the retry circuit breaker was triggered but did not dominate the overall protocol.

6.4.3 Heartbeat and Stability

The main value of Heartbeat lies in state archiving: when a new best or a phase of discards appears, the system writes the current feasible directions, rejected directions, and next-step priorities into the session. Although final-value improvement is limited, this mechanism strengthens process interpretability and provides structured evidence for long-task diagnosis.

6.5 Results Discussion

The joint interpretation of Tables 2, 3, and 4 shows that:

- The memory mechanism performs best on mean metrics, but more task validation is needed for stability;
- The deduplication mechanism is most suitable for constraining duplicate experiment inflation, and its paired results are the most stable;
- Full control records a more complete governance chain, but it needs task-adaptive parameters to suppress over-control.

7. Discussion

7.1 Why the Control Layer Is More Critical Than Simply Scaling the Model

Under fixed-model conditions, one key to improving the quality of the autonomous process is to let the agent see executable historical evidence before each proposal. In other words, the model is constrained not only by reasoning ability, but also by the quality of experiment memory and the execution boundary. The harness layer provides both and can therefore change exploration behavior without increasing model parameter scale.

7.2 Mechanism Interaction and Prompt-Injection Scheduling Competition

The fact that Full Harness does not outperform the strongest single mechanism does not mean that the three control mechanisms themselves are ineffective. Instead, it exposes a finer-grained system problem: multiple external control signals must be scheduled through the same prompt-injection strategy before entering a single Proposal Agent.

In this implementation, Memory, Dedup, and Heartbeat all run on the harness side rather than being executed by additional LLM agents. Memory retrieves historical experience through structured session records; Dedup performs deterministic interception through parameter-

signature matching; Heartbeat generates phase summaries through rule templates. The outputs of all three are eventually merged into the context-construction process for the next-round proposal. This design ensures lower cost and higher reproducibility, but it also creates a prompt-injection scheduling problem: even if the underlying LLM supports a very long context window, the current implementation still selects only a limited number of control entries when constructing the prompt, so Heartbeat summaries or Dedup retry feedback may be weakened in the final input.

Therefore, the non-additive performance of Full Harness can be interpreted as a control-signal scheduling competition rather than simple mechanism failure. When a single mechanism runs alone, the Agent receives a relatively single and dense control signal; when multiple mechanisms are enabled simultaneously, different control signals may mask one another, preventing the Agent from receiving complete history, constraints, and retry feedback stably. This indicates that autonomous experiment systems need not only state recording and governance, but also explicit prompt assembly and priority strategies.

This finding also points to future system improvements: different control signals can be split into typed prompt sections, or a priority queue can be introduced so that Dedup retry feedback, the latest best, failure forbidden regions, and long-term memory each occupy stable positions rather than relying on a unified limited-entry injection rule. In other words, the key problem in long-horizon autonomous experiments is not only how to make the system remember more, but also how to ensure that the right control signal enters model decision-making in the right round.

7.3 Engineering Significance for Reproducibility

One of the greatest values of this framework is that it builds reproducibility into the controller: 1. all proposals have unified signatures; 2. all failures can be classified; 3. summaries and history are version-traceable; 4. it does not rely on main-agent memory.

This makes experiments reproduced across teams and machines easier to audit and debug.

7.4 Risk and Threat Analysis

7.4.1 Internal Validity Risks

- The number of seeds is limited and may be affected by variance;
- The metrics depend on a single task, and task characteristics determine mechanism gains;
- The statistical tests do not cover stricter confidence-interval validation.

7.4.2 External Validity Risks

- Parameter-controlled tasks differ from free code-modification scenarios;
- The single-card budget and fixed metrics limit transferability;
- exact duplicate does not cover semantic duplication.

7.4.3 Method-Selection Risks

When task noise is high and the objective function is steep, overly aggressive deduplication may suppress neighborhood-perturbation exploration. The controller itself also needs task-adaptive parameters, such as the retry cap and summarization interval.

7.5 Integration Directions for Future Systems

The framework can be extended into a general autonomous research bus:

- Share a unified session database with multi-agent systems;
- Extend deduplication from exact duplicates to vector semantic distance;
- Introduce dynamic strategy learning, such as automatically adjusting R and the heartbeat period by task;
- Use a multi-objective decision function that combines val_bpb, stability, and cost.

8. Conclusion

Centered on the core question of why parameter-controlled autonomous experiments are inefficient in long-horizon settings, this paper proposes a three-layer control architecture and three external governance mechanisms. Empirical results show that, on the 4090D parameter-controlled compatibility task, external state-governance mechanisms change agent exploration behavior:

- Memory retrieval obtains the lowest mean best_val_bpb;
- Experiment deduplication is the most stable in seed-paired comparison and effectively intercepts duplicate parameter proposals;
- Heartbeat summarization strengthens process interpretability and long-horizon state archiving;
- Full control has governance completeness but does not show monotonic additive gain.

These results support a limited but reproducible view: in autonomous experiments where the parameter space can be formalized, the training program is fixed, and the budget is controlled,

performance is determined not only by model reasoning ability but by controllable state, reproducible execution, and interpretable updates together. In other words, improving the reliability of this type of LLM autonomous experiment should begin with a reliable controller before model expansion is considered.

Future work will focus on three directions: 1. introducing semantic-level duplicate detection into deduplication; 2. establishing adaptive strategies for control parameters, including retry caps and summarization frequency; 3. validating the transferability and multi-task robustness of the framework on more complex free code-modification tasks.

References

[1] Brown T B, Mann B, Ryder N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems. 2020, 33: 1877-1901.

[2] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[C]//Advances in Neural Information Processing Systems. 2022, 35: 27730-27744.

[3] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing reasoning and acting in language models[C]//International Conference on Learning Representations. 2023.

[4] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems. 2022, 35: 24824-24837.

[5] Wang X, Wei J, Schuurmans D, et al. Self-consistency improves chain of thought reasoning in language models[C]//International Conference on Learning Representations. 2023.

[6] Park J S, O'Brien J C, Cai C J, et al. Generative agents: Interactive simulacra of human behavior[C]//Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. New York: ACM, 2023: 1-22.

[7] Bergstra J, Bengio Y. Random search for hyper-parameter optimization[J]. Journal of Machine Learning Research, 2012, 13: 281-305.

[8] Snoek J, Larochelle H, Adams R P. Practical Bayesian optimization of machine learning algorithms[C]//Advances in Neural Information Processing Systems. 2012, 25: 2951-2959.

[9] Li L, Jamieson K, DeSalvo G, et al. Hyperband: A novel bandit-based approach to hyperparameter optimization[J]. Journal of Machine Learning Research, 2018, 18(185): 1-52.

[10] Zaharia M, Chen A, Davidson A, et al. Accelerating the machine learning lifecycle with MLflow[J]. IEEE Data Engineering Bulletin, 2018, 41(4): 39-45.

[11] Wang H, Fu T, Du Y, et al. Scientific discovery in the age of artificial intelligence[J]. Nature, 2023, 620(7972): 47-60.

[12] Lu C, Lu C, Lange R T, et al. The AI Scientist: Towards fully automated open-ended scientific discovery[EB/OL]. arXiv:2408.06292, 2024.

[13] Karpathy A. AutoResearch: AI agents running research on their own codebase[EB/OL]. GitHub, 2026. https://github.com/karpathy/autoresearch.

[14] Shinn N, Cassano F, Berman E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[C]//Advances in Neural Information Processing Systems. 2023.

[15] Packer C, Wooders S, Lin K, et al. MemGPT: Towards LLMs as Operating Systems[EB/OL]. arXiv:2310.08560, 2023.

[16] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[EB/OL]. arXiv:2305.16291, 2023.

[17] Hutter F, Kotthoff L, Vanschoren J, eds. Automated Machine Learning: Methods, Systems, Challenges[M]. Cham: Springer, 2019.